



Blazor Essencial

Criando Componentes e usando parâmetros



Um componente é uma parte independente da interface do usuário com lógica de processamento para habilitar comportamento dinâmico.

Podem ser aninhados, reutilizados, compartilhados entre projetos e usados em aplicativos **MVC e Razor Pages**.

São implementados usando uma combinação de **marcação C# e HTML** em arquivos de componente **Razor** com a extensão de arquivo **.razor**.

São renderizados em uma representação na memória do **Modelo de Objeto do Documento (DOM)** do navegador chamada *árvore de renderização*, que é usada para atualizar a interface do usuário de maneira flexível e eficiente.

Aviso.razor

```
<div class="alert alert-danger mt-4 d-flex align-items-center" role="alert">  
  <i class="bi bi-x-circle-fill me-2"></i>  
  <strong>@Mensagem</strong>  
</div>
```

Parte **HTML/Razor** → define a interface do usuário.

```
@code {  
  [Parameter]  
  public string Mensagem { get; set; }  
}
```

Bloco **@code { }** → contém a lógica C#.

Usa o padrão de nomenclatura **PascalCase** onde cada palavra começa com letra maiúscula e não há separadores : **TarefaIndividual.razor**

O nome de um componente precisa começar com um caractere maiúsculo : **Aviso.razor**



Diferença Página (@page) e componente reutilizável

1- Página

Tem a diretiva **@page**.

Pode ser acessada diretamente pelo navegador via URL.

Exemplo: **@page "/produtos"** → acessa **https://localhost:5001/produtos**

2- Componente reutilizável

Não possui a diretiva **@page**

Precisa ser usado dentro de outro componente ou página

Ex: **<Aviso Mensagem="Oi" />** → um componente que exibe uma mensagem

@ Componentes : renderização e template de projeto

Modos de renderização de componentes no **Blazor Web App** (.NET 8/9):

Static SSR → apenas HTML estático gerado no servidor, sem interatividade.

Interactive Server → interatividade via **SignalR** (*tempo real entre cliente e servidor*).

Interactive WebAssembly → roda totalmente no navegador, inclusive offline.

Interactive Auto → inicia como **Server** e tenta migrar para **WebAssembly** automaticamente quando possível.

Para as aulas desta seção → Vamos usar o template **Blazor Web App** com modo **Interactive Server** usando a opção **per page/component**

Permite gerenciar **Tarefas** (baseado no **BlazorTarefas**)

```
public class Tarefa
{
    public Guid ID { get; set; }
    public string Descricao { get; set; } = string.Empty;
    public bool Concluida { get; set; }
    public DateTime DataCriacao { get; set; }
}
```

Em **App.razor** usar a biblioteca : `<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/twitter-bootstrap/4.5.3/css/bootstrap.min.css">`

Na pasta **wwwroot/dados** criar o arquivo **tarefas.json**

Usar **HttpClient** para fazer um **GET** para a URL e converter os dados em uma tabela **HTML**

```
[
  {
    "id": "f0dcda47-1f23-4538-92e0-a47dcdcc7850",
    "descricao": "Fazer o curso de Blazor",
    "concluida": false,
    "datacriacao": "2019-08-11"
  },
  {
    "id": "d21b0f7e-a5a9-45ee-ad9b-eafd911e22fa",
    "descricao": "Estudar Blazor",
    "concluida": false,
    "datacriacao": "2019-09-05"
  }
]
```

```
<div class="alert alert-warning mt-4 d-flex align-items-center" role="alert">  
  <i class="bi bi-exclamation-triangle-fill me-2"></i>  
  <strong>@Mensagem</strong>  
</div>
```

```
@code {  
  [Parameter]  
  public string Mensagem { get; set; } = string.Empty;  
}
```

- Um parâmetro é uma **propriedade pública** de um componente que permite receber dados de outro componente (*normalmente o pai*).
- No Blazor, marcamos essa propriedade com o atributo **[Parameter]**.
- Cria uma interface de entrada para o componente, tornando-o **reutilizável e configurável**.

@ Tarefas – Componente Home.razor

Ação	Data	Descrição	Concluída
	11/08/2019 00:00:00	Fazer o curso de Blazor	☐ False
	05/09/2019 00:00:00	Estudar Blazor	☐ False

Tarefa

Registrar Tarefa

```
<table class="table">
  <thead>
    <tr>
      <th>Ação</th>
      <th>Data</th>
      <th>Descrição</th>
      <th>Concluída</th>
    </tr>
  </thead>
  <tbody>
    @foreach(var item in tarefas)...
    <tr>
      <td>button</td>
      <td>@item.DataCriacao</td>
      <td>@item.Descricao</td>
      <td>...item.Concluida</td>
    </tr>
    ...
    ...
  </tbody>
</table>
```

```
@if (tarefas != null)
{
  <div class="form-group">
    <label>Tarefa</label>
    <input type="Text" placeholder="Informe a tarefa" @bind="novaTarefa" />
    <button class="btn btn-primary" @onclick="AdicionarNovaTarefa">Registrar
Tarefa</button>
  </div>
}
```